

• Medium → #39 • Combination Sum

▲ Problem Statement → Given an array of distinct integers "candidates" & a "target", find all unique combinations of candidates that add up to the target.

Each number can be chosen unlimited times.

▲ Main Problem Solution → Use "backtracking" to explore all possible combination.

For every candidate, we have two choices :-

- Take it → We can use the same candidate again.
- Skip it → Move to the next candidate.

When the remaining target becomes "0", we have found a valid combination.

- ▲ Algorithm → 1. Create an empty "result" & "current" combination.
2. Start backtracking from index "0".
3. If "target == 0", add the current combination to "result".
4. If "target < 0", stop that path.
5. Loop through candidates starting from the current index.
6. Choose the current candidate & add it to "current".
7. Call backtracking with the same index because the no. can be reused.
8. Remove the candidates after returning from recursion.
9. Continue with the next candidate.
10. Return all valid combinations.

▲ Important Points → This problem uses backtracking.

- "current.push_back()" → choose a candidate
- "current.pop_back()" → undo the choice
- pass "i" instead of "i+1" because candidates can be reused.

- "target == 0" → valid combination
- "target < 0" → Invalid path
- Starting the loop from "start" prevents duplicate combinations in different orders.

▲ Complexity → Time → $O(2^{\text{target}})$
 Space → $O(\text{target})$